

Sistema distribuido para gestión de estado de usuarios

Juan Cruz
Telecomunicaciones y Sistemas Distribuidos

2025

Índice

1	Introducción	2
1.1	Enunciado del problema	2
1.2	Solución elegida: Chord	2
2	Arquitectura del sistema	2
2.1	Red de overlay vs. red física	2
2.2	Estructura de un nodo	2
2.3	El anillo	3
3	Algoritmos	3
3.1	Hashing consistente	3
3.2	Búsqueda: <code>find_successor</code>	3
3.3	Protocolo de Join	4
3.4	Estabilización	4
3.5	Tolerancia a fallas	4
4	Formato de mensajes	5
4.1	Transporte	5
4.2	Catálogo de mensajes	5
5	Herramientas utilizadas	6
6	Demostración de ejecución	6
6.1	Formación del anillo	6
6.2	Escritura desde nodos arbitrarios	7
6.3	Incorporación de un nodo extra	8
6.4	Caída del nodo dueño de una clave	9
6.5	Continuidad del sistema	10
6.6	Propiedades verificadas	10
7	Demostración de propiedades	11

1 Introducción

1.1 Enunciado del problema

Se requiere una aplicación a escala global que mantenga el estado de conexión (conectado/desconectado) de los usuarios. Los clientes deben poder conectarse a *cualquier* nodo del sistema para consultar o actualizar dicho estado. Se imponen tres requisitos:

1. Los datos deben estar **uniformemente distribuidos** entre los nodos de la red.
2. Las consultas deben responder con un costo **a lo sumo logarítmico**, respecto de la cantidad de mensajes.
3. El sistema debe ser **tolerante a fallas**: ante la caída de un nodo, no debe perderse información sobre los usuarios.

1.2 Solución elegida: Chord

De los algoritmos de tablas de hash distribuidas (DHT) sugeridos en el enunciado, se optó por **Chord** por dos razones principales:

- Su estructura de anillo con *finger tables* admite una demostración directa de la cota $O(\log N)$ sobre la cantidad de saltos, que se corresponde de forma natural con el requisito 2 del enunciado.
- La separación entre el mecanismo de enrutamiento (finger table) y el mecanismo de tolerancia a fallas (lista de sucesores y replicación) permite razonar y probar cada propiedad de manera independiente.

2 Arquitectura del sistema

2.1 Red de overlay vs. red física

Cada nodo es un proceso independiente (potencialmente en una máquina distinta) identificado por su dirección IP:puerto. Sobre la red física (en la que, en principio, cualquier nodo podría contactar a cualquier otro) se construye una **red lógica de overlay (anillo)**: cada nodo conoce únicamente un conjunto acotado de referencias a otros nodos (sucesor, predecesor, finger table y lista de sucesores), en lugar de conocer a la totalidad de los N nodos.

2.2 Estructura de un nodo

Cada nodo mantiene:

- **id**: resultado de aplicar SHA-1 a su dirección, módulo 2^m .
- **successor** / **predecesor**: referencias (id, dirección) a sus vecinos inmediatos en el anillo.
- **successor_list**: lista de los próximos r sucesores, usada para tolerancia a fallas.
- **finger_table**: m entradas, la entrada i apunta al sucesor de $(\text{id} + 2^i) \bmod 2^m$.
- **data**: diccionario local $\text{clave} \rightarrow (\text{usuario}, \text{estado})$ de las claves que le pertenecen.
- **replicas**: copias de respaldo de claves de otros nodos (aquellos de los cuales este nodo es sucesor).

2.3 El anillo

La Figura 1 ilustra la estructura conceptual: los nodos ocupan posiciones en un anillo de tamaño 2^m según su id hashado; una clave pertenece al primer nodo cuyo id es mayor o igual a ella, recorriendo el anillo en sentido horario (su sucesor).

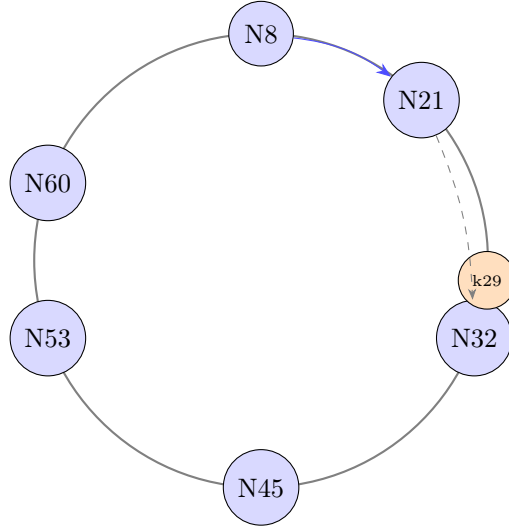


Figura 1: Anillo de identificadores (espacio $[0, 64]$ a modo de ejemplo). La clave **k29** pertenece a **N32**, su sucesor. La flecha azul sólida es el único mensaje de red que hace falta: **N8** le pregunta a **N21** (su mejor salto de finger table). La flecha gris punteada (no es un mensaje adicional) representa que **N21**, sin poseer la clave, ya sabe localmente que **k29** cae entre él y su propio sucesor (**N32**), y devuelve esa referencia sin una segunda consulta por red.

3 Algoritmos

3.1 Hashing consistente

Tanto los nodos (por su dirección) como las claves (por el nombre de usuario) se hashan con **SHA-1** al mismo espacio $[0, 2^m]$. Esto garantiza que, en promedio, cada nodo reciba una fracción similar del espacio total de claves ($1/N$), cumpliendo el requisito de distribución uniforme de los datos, y que la incorporación o salida de un nodo solo afecte a una fracción pequeña de las claves (las de sus vecinos inmediatos), no a la totalidad del sistema.

3.2 Búsqueda: `find_successor`

El Listado 1 muestra el algoritmo central de Chord. Un nodo que no puede responder directamente reenvía la consulta al nodo conocido más cercano (pero anterior) al identificador buscado, usando su finger table. Esto es lo que produce la cota logarítmica de saltos (demostrada en la Sección 7).

Listing 1: Pseudocódigo de `find_successor`

```
function find_successor(id):
    if id in (self.id, successor.id):
        return successor
    for candidate in closest_preceding_candidates(id): # de mas lejano a mas cercano
        try:
            return RPC(candidate, "find_successor", id)
```

```

except ConnectionError:
    continue # candidato caído, probar el siguiente
for succ in successor_list: # ultimo recurso
    try:
        return RPC(succ, "find_successor", id)
    except ConnectionError:
        continue
return self

```

3.3 Protocolo de Join

Cuando un nodo nuevo n se incorpora (Listado 2): Calcula su `id` y localiza su sucesor llamando a `find_successor(n.id)` a través de un nodo de entrada conocido (cualquier nodo ya perteneciente al anillo). Su predecesor al momento de unirse al anillo es *null*, que luego será actualizado tras una pasada de estabilización (`stabilize()`).

Listing 2: Pseudocódigo de join.

```

function join(known_node):
    predecessor = null
    successor = RPC(known_node, "find_successor", self.id)

```

3.4 Estabilización

Dado que el join solo actualiza los punteros de los vecinos inmediatos, el resto del anillo se entera del cambio de forma asincrónica mediante tres procesos periódicos, cada uno en su propio hilo y tiempo:

- `stabilize()`: cada nodo le pregunta a su sucesor quién es *su* predecesor; si resulta ser un nodo intermedio no conocido, lo adopta como nuevo sucesor. También actualiza su `successor_list`.
- `notify(candidate)`: un nodo se envía (a si mismo) como **candidate** a otro nodo (quien cree que es su nuevo sucesor) presentándose como su candidato a predecesor. Y lo será si, el otro nodo no tiene uno, o si **candidate** está más cerca que el actual.
- `fix_fingers()`: actualiza, de a una posición $(0, 1, \dots i+1)$ por vez y en forma rotativa, una entrada de la finger table.

3.5 Tolerancia a fallas

Para lograr esta propiedad, se combinan tres mecanismos:

1. **Lista de sucesores** de tamaño r (parámetro `num_replicas`): en vez de conocer solo al sucesor inmediato, cada nodo conoce a los próximos r . Si el nodo falla, las claves que le pertenecían pasan a ser del siguiente de la lista, sin necesidad de reconstruir el anillo desde cero.
2. **Replicación**: al guardar una clave, el dueño la replica de inmediato en los r nodos de su `successor_list`. En total se mantienen (contando su propia copia) $r + 1$ copias de cada clave.

3. **Detección de fallas y promoción de réplicas:** cada nodo hace *ping* periódico a su predecesor (`check_predecessor()`). Si no responde luego de 3 heartbeats, lo descarta y promueve como datos primarios las réplicas que tuviera de las claves de ese nodo que ya no responde, ya que tras la estabilización del anillo, pasa a ser el nuevo dueño legítimo de ese rango.

4 Formato de mensajes

4.1 Transporte

La comunicación entre nodos se implementa como RPC sobre sockets TCP, usando **JSON delimitado por salto de línea** como formato de mensaje: cada request y cada response es un objeto JSON en una única línea. Este formato se eligió por ser simple, liviano y legible para el alcance de este problema.

Listing 3: Formato de un request y su response.

```
// Request (cliente -> nodo)
{"method": "find_successor", "args": [40]}

// Response (nodo -> cliente), caso exitoso
{"result": {"__noderef__": true, "address": "127.0.0.1:9001", "id": 122}}

// Response, caso de error
{"error": "ConnectionError: Node at 127.0.0.1:9003 is unreachable"}
```

Las referencias a nodos (NodeRef) se serializan con una marca `__noderef__` para poder reconstruir el objeto correcto del lado receptor sin ambigüedad.

4.2 Catálogo de mensajes

Todo mensaje del sistema es una invocación remota de uno de estos métodos.

Método	Argumentos	Retorno
<code>find_successor</code>	id (entero), hops (entero, opcional)	NodeRef del nodo dueño
<code>get_predecessor</code>	—	NodeRef o null
<code>get_successor_list</code>	—	lista de NodeRef
<code>notify</code>	NodeRef candidato	—
<code>ping</code>	—	true
<code>write</code>	usuario, estado	— (enruta al dueño)
<code>read</code>	usuario	estado o null
<code>write_local</code>	usuario, estado	— (guarda y replica)
<code>read_local</code>	usuario	estado o null
<code>receive_keys</code>	lista de (clave, usuario, estado)	—
<code>store_replica</code>	usuario, estado, NodeRef dueño	—
<code>drop_replica</code>	lista de usuarios, NodeRef dueño	—
<code>get_debug_state</code>	—	snapshot del estado del nodo

Los siguientes métodos son *locales*: no están expuestos por RPC y nunca cruzan la red. Se ejecutan por temporizador o como consecuencia de otra operación del mismo nodo, y son los que emiten los mensajes del catálogo anterior.

Método	Se ejecuta
<code>join</code>	Una vez, al arrancar el nodo
<code>stabilize</code>	Cada 1 s
<code>fix_fingers</code>	Cada 1 s
<code>check_predecessor</code>	Cada 1 s
<code>hand_off_keys</code>	Al ejecutar <code>stabilize</code>
<code>sync_replicas</code>	Al ejecutar <code>stabilize</code>
<code>replicate</code>	Desde <code>write_local</code> , <code>receive_keys</code> y la promoción
<code>closest_preceding_candidates</code>	Desde <code>find_successor</code>
<code>read_from_replica_holders</code>	Desde <code>read</code> , si el dueño no responde

5 Herramientas utilizadas

- **Lenguaje:** Python 3.
- **Hashing:** `hashlib.sha1` (módulo estándar), con $m = 160$.
- **Transporte de red:** sockets TCP crudos (`socket/socketserver`).
- **Serialización:** JSON (módulo `json` estándar), con un `JSONEncoder` y un `object_hook` propios para serializar/deserializar referencias a nodos.
- **Concurrencia:** `threading` — un hilo para el servidor RPC (con manejo concurrente de conexiones vía `ThreadingTCPServer`) y otro para los procesos periódicos de mantenimiento (`stabilize`, `fix_fingers`, `check_predecessor`).

6 Demostración de ejecución

A continuación, se muestra una ejecución completa con siete nodos, cada uno como proceso independiente en su propia terminal. Cada proceso informa por su cuenta los eventos relevantes del protocolo, de modo que el comportamiento del anillo se observa y deduce directamente a través de las terminales.

Los nodos se identifican por su puerto y por los últimos seis dígitos hexadecimales de su identificador de 160 bits, que es la forma en que aparecen en los registros.

6.1 Formación del anillo

El primer nodo crea el anillo uniéndose a sí mismo. Los siguientes ingresan a través de un nodo cualquiera que ya sea miembro.

Terminal 1 — `python3 server.py --port 9000`

```
[127.0.0.1:9000] listening (id=..cbfba5)
[127.0.0.1:9000] creating new ring
[127.0.0.1:9000] predecessor -> ..cbfba5
[127.0.0.1:9000] predecessor -> ..64e67a
[127.0.0.1:9000] successor -> ..64e67a
[127.0.0.1:9000] successor -> ..c9a4a9
[127.0.0.1:9000] predecessor -> ..e539e1
[127.0.0.1:9000] successor -> ..3febfd
```

El nodo inicial se "une" a sí mismo, por lo que queda siendo su propio sucesor y su propio predecesor (`predecessor -> ..cbfba5`). Las líneas siguientes son el protocolo de estabilización corrigiendo los punteros a medida que ingresan nodos más cercanos; el sucesor se revisa cada segundo y va cambiando de `..64e67a` a `..c9a4a9` y finalmente a `..3febfd`.

Terminales 2 a 4

```
$ python3 server.py --port 9001 --join 127.0.0.1:9000
[127.0.0.1:9001] listening (id=..64e67a)
[127.0.0.1:9001] joined ring through 127.0.0.1:9000 -> successor=..cbfba5
[127.0.0.1:9001] predecessor -> ..cbfba5
[127.0.0.1:9001] predecessor -> ..c9a4a9
[127.0.0.1:9001] predecessor -> ..4772f3
[127.0.0.1:9001] successor -> ..e539e1

$ python3 server.py --port 9002 --join 127.0.0.1:9000
[127.0.0.1:9002] listening (id=..c9a4a9)
[127.0.0.1:9002] joined ring through 127.0.0.1:9000 -> successor=..64e67a
[127.0.0.1:9002] predecessor -> ..cbfba5
[127.0.0.1:9002] successor -> ..4772f3
[127.0.0.1:9002] predecessor -> ..3febfd
[127.0.0.1:9002] predecessor -> ..f9988d

$ python3 server.py --port 9003 --join 127.0.0.1:9002
[127.0.0.1:9003] listening (id=..4772f3)
[127.0.0.1:9003] joined ring through 127.0.0.1:9002 -> successor=..64e67a
[127.0.0.1:9003] predecessor -> ..c9a4a9
```

Terminales 5 a 7

```
$ python3 server.py --port 9004 --join 127.0.0.1:9002
[127.0.0.1:9004] listening (id=..e539e1)
[127.0.0.1:9004] joined ring through 127.0.0.1:9002 -> successor=..cbfba5
[127.0.0.1:9004] predecessor -> ..64e67a

$ python3 server.py --port 9005 --join 127.0.0.1:9002
[127.0.0.1:9005] listening (id=..3febfd)
[127.0.0.1:9005] joined ring through 127.0.0.1:9002 -> successor=..c9a4a9
[127.0.0.1:9005] predecessor -> ..cbfba5
[127.0.0.1:9005] successor -> ..f9988d

$ python3 server.py --port 9008 --join 127.0.0.1:9000
[127.0.0.1:9008] listening (id=..f9988d)
[127.0.0.1:9008] joined ring through 127.0.0.1:9000 -> successor=..c9a4a9
[127.0.0.1:9008] predecessor -> ..3febfd
```

Los nodos :9003, :9004 y :9005 ingresan a través de :9002, no del nodo inicial. Esto evidencia que el nodo creador del anillo no cumple ningún rol privilegiado: su única función es responder la primera consulta de ubicación y permitir que más nodos se unan a través de él, luego es un nodo como cualquier otro. Cualquier miembro sirve como punto de entrada al anillo.

El orden resultante, determinado por los identificadores (quienes surgen de aplicar la función de hash consistente) y no por el orden de ingreso ni por los puertos, es:

:9003 → :9001 → :9004 → :9000 → :9005 → :9008 → :9002 → :9003

6.2 Escritura desde nodos arbitrarios

Cada escritura se hace desde un nodo cualquiera, que en general no es el responsable de la clave. El nodo, a través del cual se ejecuta el `write`, calcula el hash del usuario, resuelve quién es el dueño y reenvía la operación. Como se ve a continuación, en el `write('juan', 'connected')` (que se hace desde :9004), se calcula el hash para el username y se reenvía al nodo que le corresponde (:9008) para que ejecute el `write_local`.

Cliente y terminales involucradas

```
$ python3 -c "net.send('127.0.0.1:9004', 'write', 'juan', 'connected')"
[127.0.0.1:9004] cliente solicita write(juan) -> reenviando a owner ..f9988d
[127.0.0.1:9008] write_local: juan -> connected

$ python3 -c "net.send('127.0.0.1:9004', 'write', 'agustin', 'connected')"
[127.0.0.1:9004] cliente solicita write(agustin) -> reenviando a owner ..cbfba5
[127.0.0.1:9000] write_local: agustin -> connected

$ python3 -c "net.send('127.0.0.1:9001', 'write', 'juan', 'disconnected')"
[127.0.0.1:9001] cliente solicita write(juan) -> reenviando a owner ..f9988d
[127.0.0.1:9008] write_local: juan -> disconnected

$ python3 -c "net.send('127.0.0.1:9008', 'write', 'maria', 'connected')"
[127.0.0.1:9008] cliente solicita write(maria) -> reenviando a owner ..4772f3
[127.0.0.1:9003] write_local: maria -> connected
```

Cada par de líneas exhibe la separación entre enrutamiento y almacenamiento: el nodo que atiende al cliente informa a quién reenvía, y el dueño informa que guardó el dato. Son dos nodos distintos en los cuatro casos.

Las dos escrituras de **juan** ingresan por nodos diferentes (:9004 y luego :9001) y ambas resuelven al mismo dueño **..f9988d**. La resolución no depende del punto de entrada: se calcula a partir del hash del nombre de usuario, de forma idéntica en todos los nodos.

Estado del anillo tras las cuatro escrituras:

Nodo	Identificador	Dueño de	Respalda
:9003	..4772f3	maria	juan
:9001	..64e67a	—	maria
:9004	..e539e1	—	maria
:9000	..cbfba5	agustin	—
:9005	..3febfd	—	agustin
:9008	..f9988d	juan	agustin
:9002	..c9a4a9	—	juan

Cada clave reside en tres nodos: su dueño y sus dos sucesores inmediatos. Por ejemplo **maria** es propiedad de :9003 y está respaldada en :9001 y :9004, que son precisamente los dos nodos que le siguen en el anillo.

Con cuatro claves y siete nodos, tres de éstos no son dueños de ninguna. Esto es esperable y obvio, ya que cada clave solo tiene un único dueño. El hashing consistente garantiza que al incorporarse o retirarse un nodo sólo se reasignen K/N claves, pero no asegura un reparto uniforme para valores pequeños de K .

6.3 Incorporación de un nodo extra

Se levanta un octavo nodo sobre el anillo ya en funcionamiento y con datos almacenados.

Alta de :9006 y reacción de :9008

```
[127.0.0.1:9006] listening (id=..2fed78)
[127.0.0.1:9006] joined ring through 127.0.0.1:9000 -> successor=..c9a4a9
[127.0.0.1:9006] predecessor -> ..f9988d
[127.0.0.1:9008] successor -> ..2fed78
[127.0.0.1:9008] ..2fed78 entro a mi successor list -> replicando 1 clave(s)
[127.0.0.1:9008] ..4772f3 salio de mi successor list -> descartando mis replicas ahi
```

El nodo nuevo se inserta entre :9008 y :9002. Las dos últimas líneas corresponden al mecanismo de reconciliación de réplicas, y muestran sus dos ramas en un mismo evento:

- `..2fed78 (:9006)` *ingresó* a la lista de sucesores de `:9008`, por lo que recibe una copia de las claves que `:9008` ya poseía. La replicación sólo se dispara al escribir, de modo que sin este mecanismo el nodo nuevo nunca recibiría las claves preexistentes.
- `..4772f3 (:9003)` *salió* de esa lista, desplazado a la tercera posición. Sigue vivo, pero ya no le corresponde respaldar las claves de `:9008`, por lo que se le indica descartarlas. De lo contrario acumularía copias que nadie mantiene ni consulta.

El estado del anillo ahora es: `juan` pasa a estar respaldado en `:9006` y `:9002`, y desaparece de `:9003`.

Nodo	Identificador	Dueño de	Respalda
<code>:9003</code>	<code>..4772f3</code>	<code>maria</code>	—
<code>:9001</code>	<code>..64e67a</code>	—	<code>maria</code>
<code>:9004</code>	<code>..e539e1</code>	—	<code>maria</code>
<code>:9000</code>	<code>..cbfba5</code>	<code>agustin</code>	—
<code>:9005</code>	<code>..3febfd</code>	—	<code>agustin</code>
<code>:9008</code>	<code>..f9988d</code>	<code>juan</code>	<code>agustin</code>
<code>:9006</code>	<code>..2fed78</code>	—	<code>juan</code>
<code>:9002</code>	<code>..c9a4a9</code>	—	<code>juan</code>

Ninguna clave cambió de dueño en esta operación: el tramo del anillo que tomó `:9006` no contenía ninguna de las cuatro claves almacenadas.

6.4 Caída del nodo dueño de una clave

Se produce una caída del proceso `:9003`, dueño de `maria`:

Reacción de los nodos vecinos

```
[127.0.0.1:9001] predecessor ..4772f3 caido -> promoviendo 1 clave(s) de replica a dato primario
[127.0.0.1:9001] predecessor -> ..c9a4a9
[127.0.0.1:9002] successor ..4772f3 caido -> promoviendo successor ..64e67a
```

Intervienen dos mecanismos distintos, en nodos distintos y sin coordinación entre sí:

- `:9001`, sucesor del nodo caído, detecta mediante `check_predecessor` que su predecesor dejó de responder y asciende a dato primario la réplica de `maria` que ya mantenía. Esto no requiere mover datos por la red: la copia estaba en el lugar correcto de antemano, ya que se colocan (a propósito) en los sucesores del dueño. Así, nadie se enteró de la caída del nodo y todo siguió funcionando normal (los datos siguen disponibles y se pueden consultar).
- `:9002`, predecesor del nodo caído, detecta mediante `stabilize` que su sucesor no responde y promueve al siguiente candidato de su lista de sucesores, con lo que la cadena del anillo vuelve a cerrarse.

Se puede observar que, `:9001` promueve *una sola* clave y no todas las réplicas que mantiene: cada réplica registra a qué nodo pertenece, y sólo se ascienden las del nodo que efectivamente falló. Promover las demás asignaría a `:9001` claves cuyo dueño sigue en funcionamiento, dejando dos nodos respondiendo por el mismo usuario.

Nodo	Identificador	Dueño de	Respalda
:9001	..64e67a	maria	—
:9004	..e539e1	—	maria
:9000	..cbfba5	agustin	maria
:9005	..3febfd	—	agustin
:9008	..f9988d	juan	agustin
:9006	..2fed78	—	juan
:9002	..c9a4a9	—	juan

maria pasó a :9001 y volvió a tener tres copias: el nuevo dueño más :9004 y :9000, sus dos sucesores. La redundancia se restituyó automáticamente.

6.5 Continuidad del sistema

La lectura se realiza contra un nodo que no es dueño ni tenedor de réplica de la clave consultada:

```
$ python3 -c "print(net.send('127.0.0.1:9005', 'read', 'juan'))"
disconnected
```

El valor recuperado es `disconnected`, correspondiente a la *segunda* escritura de `juan`, lo que confirma que las actualizaciones sobre una clave existente reemplazan el valor anterior y que la lectura es coherente con la última escritura efectuada.

Una escritura posterior demuestra que el anillo, ya reconfigurado tras el alta de un nodo y la baja de otro, sigue funcionando con normalidad:

```
[127.0.0.1:9004] cliente solicita write(matias) -> reenviando a owner ..64e67a
[127.0.0.1:9001] write_local: matias -> connected
```

Nodo	Identificador	Dueño de	Respalda
:9001	..64e67a	maria, matias	—
:9004	..e539e1	—	maria, matias
:9000	..cbfba5	agustin	maria, matias
:9005	..3febfd	—	agustin
:9008	..f9988d	juan	agustin
:9006	..2fed78	—	juan
:9002	..c9a4a9	—	juan

6.6 Propiedades verificadas

La corrida documentada evidencia las tres propiedades requeridas:

1. **Distribución de claves entre nodos.** Las cuatro claves quedaron repartidas entre cuatro nodos distintos a lo largo de la corrida (:9003, :9000, :9008 y :9001), determinado únicamente por el hash de cada nombre de usuario y sin ninguna tabla de asignación centralizada.
2. **Tolerancia a fallas sin pérdida de datos.** Tras la caída del nodo dueño de `maria`, la clave siguió disponible y recuperó sus tres copias de manera autónoma. Los nodos reorganizaron el anillo y redistribuyeron las réplicas en segundos.
3. **Acceso desde cualquier nodo.** Las escrituras ingresaron por :9004, :9001 y :9008, y la lectura por :9005. En todos los casos el nodo receptor resolvió el responsable y reenvió la operación. Las dos escrituras de `juan`, ingresadas por nodos distintos, resolvieron al mismo dueño.

7 Demostración de propiedades

Teorema 1 (Costo de búsqueda logarítmico). *En un anillo de N nodos con finger tables correctamente pobladas, `find_successor` requiere $O(\log N)$ mensajes en el caso esperado.*

Idea de la demostración. Sea n el nodo que origina la búsqueda del identificador id , y sea d la distancia circular (en sentido horario) entre n y id . La entrada i de la finger table de n apunta al sucesor de $(n + 2^i) \bmod 2^m$. Dado que las entradas cubren distancias que crecen exponencialmente $(1, 2, 4, \dots, 2^{m-1})$, el salto elegido (el de mayor índice i tal que `finger[i]` precede a id) reduce la distancia a, como máximo, la mitad de d en cada paso. Como la distancia inicial está acotada por 2^m , y se reduce a la mitad en cada salto, se necesitan a lo sumo $\log_2(2^m) = m$ saltos en el peor caso, y en la práctica, dado que solo hay $N \leq 2^m$ nodos, el número de saltos es $O(\log N)$. $\square$

Teorema 2 (Tolerancia a fallas sin pérdida de datos). *Sea k una clave con dueño y lista de sucesores $s_1, \dots, s_r$ (de modo que existen $r + 1$ copias de k : una primaria y r réplicas). Si a lo sumo r de estos $r + 1$ nodos fallan (no necesariamente simultáneamente), k no se pierde.*

Justificación. Por hipótesis, al menos uno de los $r + 1$ nodos que poseen una copia de k permanece activo. Cuando un nodo cae, su antiguo vecino (quien lo tenía como predecesor) lo detecta mediante `check_predecessor()` y, si sostenía una réplica de las claves del nodo caído, la promueve a dato primario. Como el proceso de `stabilize()` por otro lado, reconfigura los punteros del anillo para que ese vecino pase a ser el nuevo responsable legítimo del rango de claves que antes pertenecía al nodo caído, el resultado es consistente: la clave sigue siendo accesible desde cualquier punto de entrada, con un único dueño luego de la reconfiguración. $\square$

Se puede observar que esta propiedad se cumplió en la demostración de ejecución realizada más arriba. Incluso se cumple para el caso de que el nodo caído fuera un salto intermedio (no el dueño final) de una búsqueda en curso, donde `find_successor` reintenta con el siguiente candidato de la finger table o, en última instancia, con la `successor_list`.

References

- [1] M. Arroyo. *Telecomunicaciones y Sistemas Distribuidos — Notas de curso*. Universidad Nacional de Río Cuarto, 2025.
- [2] I. Stoica, R. Morris, D. Karger, M. F. Kaashoek, H. Balakrishnan. *Chord: A Scalable Peer-to-peer Lookup Service for Internet Applications*. ACM SIGCOMM, 2001.